Web后端知识

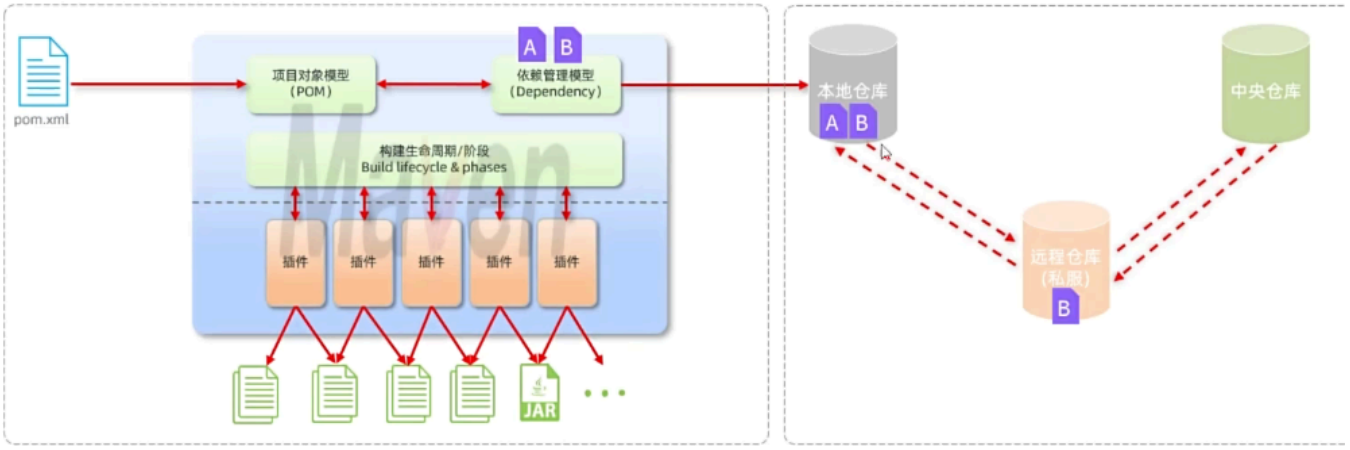
一.Maven

1.1 Maven相关介绍

Maven是用来管理和构建Java项目的工具

- 方便快捷地管理项目依赖的资源(jar包)
- 标准化的跨平台自动化项目构建方式
- 提供标准，统一的项目结构

Maven结构



Maven的仓库用来存储和管理jar包

分为本地仓库，远程仓库和中央仓库

Maven坐标

- maven中的坐标是资源(jar)的唯一标识，通过该坐标可以唯一定位资源位置
- 使用坐标来定义项目或引入项目中需要的依赖

- Maven坐标的主要组成
  - groupId:定义当前Maven项目隶属组织的名称（域名反写）
  - artifactId:定义当前Maven项目名称（通常是模块名称 如order-service,goods-service)
  - version:当前版本号

1.2 Maven依赖管理

- 在pom.xml中编写标签
- 在标签中使用引入坐标

- 定义坐标的groupId,artifactId,version
- 刷新，引入最新加入的坐标

1.3 单元测试

二.Web基础

2.1 SpringBoot入门

入门程序的剖析

起步依赖

- spring-boot-starter-webmvc-test：包含单元测试的需要的所有依赖
- spring-boot-starter-webmvc:包含web开发的所有依赖

运行main方法运行内部集成的tomcat服务器（依赖中包含的方法）

2.2 HTTP协议

- 超文本传输协议，浏览器与服务器之间数据传输的规则
- 基于TCP协议：面向连接，安全
- 基于请求-响应模型：一次请求对应一次响应
- HTTP协议是无状态的协议：对于事务处理没有记忆能力，每次的请求-响应是独立的
    - 缺点：多次请求之间无法共享数据
    - 优点：速度快
- URL:用来定位和访问资源的地址，平常说的“网址”就是一种 URL。
    - 比如：http://localhost:8080/hello?name=heima

部分	含义
`http://`	使用 HTTP 协议
`localhost`	访问本机
`8080`	访问本机的 8080 端口
`/hello`	URL 路径，指定要访问的接口
`?name=heima`	查询参数，传入 `name`，值为 `heima`

2.2.1 HTTP请求协议

请求行

请求数据第一行（请求方式，资源路径，协议）

请求头

第二行开始，格式key:value

```
POST /brand HTTP/1.1
Accept: application/json, text/plain, */*
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Content-Length: 161
Content-Type: application/json;charset=UTF-8
Cookie: Idea-8296eb32=841b16f0-0cfe-495a-9cc9-d5aaa71501a6; JSESSIONID=0FDE4E430876BD9C5C955F061207386F
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/...

{"status":1,"brandName":"黑马","companyName":"黑马程序员","id":"","description":"黑马程序员"}
```

各个请求头的含义

Host	请求的主机名
User-Agent	浏览器版本，例如Chrome浏览器的标识类似Mozilla/5.0 ... Chrome/79，IE浏览器的标识类似Mozilla/5.0 (Windows NT ...) like Gecko
Accept	表示浏览器能接收的资源类型，如text/*，image/*或者*/*表示所有；
Accept-Language	表示浏览器偏好的语言，服务器可以据此返回不同语言的网页；
Accept-Encoding	表示浏览器可以支持的压缩类型，例如gzip，deflate等。
Content-Type	请求主体的数据类型。
Content-Length	请求主体的大小（单位：字节）。

请求体

POST请求，存放请求参数；请求头和请求体之间用一个空行隔开

- 对GET请求来说：请求参数在请求行中，没有请求体，GET请求大小在浏览器中是有限制的
- 对POST请求来说：请求参数在请求体中，POST请求大小是没有限制的

2.2.2 HTTP协议请求数据获取

- HttpServletRequest对象里面封装了所有的请求信息

2.2.3 HTTP响应协议

响应数据分为三个部分：

响应行，响应头，响应体

- 响应行：响应数据的第一行（协议,状态码，描述）
- 响应头：第二行开始 格式key:value
- 响应体：最后一部分，存放响应数据

```
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 10 May 2022 07:51:07 GMT
Keep-Alive: timeout=60
Connection: keep-alive

[{id: 1, brandName: "阿里巴巴",companyName: "腾讯计算机系统有限公司", description: "玩玩玩"}]
```

常见响应状态码

1xx	响应中-临时状态码，表示请求已经接收，告诉客户端应该继续请求或者如果它已经完成则忽略它。
2xx	成功-表示请求已经被成功接收，处理已完成。
3xx	重定向-重定向到其他地方；让客户端再发起一次请求以完成整个处理。
4xx	客户端错误-处理发生错误，责任在客户端。如：请求了不存在的资源、客户端未被授权、禁止访问等。
5xx	服务器错误-处理发生错误，责任在服务端。如：程序抛出异常等。

状态码	英文描述	解释
200	OK	客户端请求成功，即**处理成功**，这是我们最想看到的状态码
302	Found	指示所请求的资源已移动到由Location响应头给定的 URL，浏览器会自动重新访问
304	Not Modified	告诉客户端，你请求的资源至上次取得后，服务端并未更改，你直接用你本地缓存
400	Bad Request	客户端请求有**语法错误**，不能被服务器所理解
403	Forbidden	服务器收到请求，但是**拒绝提供服务**，比如：没有权限访问相关资源
404	Not Found	**请求资源不存在**，一般是URL输入有误，或者网站资源被删除了
405	Method Not Allowed	请求方式有误，比如应该用GET请求方式的资源，用了POST
428	Precondition Required	**服务器要求有条件的请求**，告诉客户端要想访问该资源，必须携带特定的请求头
429	Too Many Requests	指示用户在给定时间内发送了**太多请求**（“限速”），配合 Retry-After(多长时间后可
431	Request Header Fields Too Large	**请求头太大**，服务器不愿意处理请求，因为它的头部字段太大。请求可以在减少请
500	Internal Server Error	**服务器发生不可预期的错误**。服务器出异常了，赶紧看日志去吧
503	Service Unavailable	**服务器尚未准备好处理请求**，服务器刚刚启动，还未初始化好

常见响应头的含义

Content-Type	表示该响应内容的类型，例如text/html，application/json。
Content-Length	表示该响应内容的长度（字节数）。
Content-Encoding	表示该响应压缩算法，例如gzip。
Cache-Control	指示客户端应如何缓存，例如max-age=300表示可以最多缓存300秒。
Set-Cookie	告诉浏览器为当前页面所在的域设置cookie。

2.2.4 HTTP响应数据设置

方法一

通过HttpServletResponse对象封装HTTP原始数据进行解析后的结果，可以直接调用API进行操作

通过调用该对象中的方法实现相应状态码（一般无需设置，浏览器处理后自动返回），响应头和响应体

方法二

通过ResponseEntity中的方法来完成配置

方式一：基于HttpServletResponse封装

```
@RequestMapping("/response")
public void response(HttpServletResponse response) throws IOException {
    // 1.设置响应状态码
    response.setStatus(401);
    // 2.设置响应头
    response.setHeader("itheima","itheima");
    // 3.设置响应体
    response.getWriter().write("<h1>Hello Response</h1>");
}
```

方式二：基于ResponseEntity封装

```
@RequestMapping("/response")
public ResponseEntity<String> response() {
    return ResponseEntity.status(401)          // 1.设置响应状态码
            .header("group", "itcast")         // 2.设置响应头
            .body("<h1>Hello Response</h1>"); // 3.设置响应体
}
```

2.3 分层解耦

2.3.1 三层架构

Controller:控制层，接受前端发送的请求，对请求进行处理，并响应数据

service:业务逻辑层，处理具体的业务逻辑

dao：数据访问层（Data Access Object）（持久层），负责数据访问操作，包括数据的增删改查

前端给请求->controller->调用Service->调用dao取得数据

dao的实现可以有文本，网络，数据库

2.3.2 IOC与DI入门

耦合：衡量软件中各个层/各个模块的依赖关联程度

内聚：软件中各个功能模块内部的功能联系

控制反转：IOC

- 对象的创建控制权由程序自身转移到外部（容器）这里就是Spring容器
- 可以使用`@Component`注解将类交给IOC容器管理（是加在实现类上面而不是接口上面）
  - `@Component`的衍生注解：

注解	说明	位置
@Component	声明bean的基础注解	不属于以下三类时，用此注解
@Controller	@Component的衍生注解	标注在控制层类上
@Service	@Component的衍生注解	标注在业务层类上
@Repository	@Component的衍生注解	标注在数据访问层类上（由于与mybatis整合，用的少）

依赖注入：DI

- 容器为应用程序提供运行时所依赖的资源
- 具体是指当一个对象需要使用另一个对象时，由容器将它需要的对象传入。依赖注入是实现控制反转的一种方式。
- 使用`@Autowired`注解可以从IOC容器中找到该类型的bean，然后完成依赖注入

Bean对象：IOC容器中创建，管理的对象

三.MySQL数据模型

- 关系型数据库：建立在关系模型基础上，由多张相互连接的二维表组成的数据库

3.1 SQL语句-DDL

- SQL：一门操作关系型数据库的编程语言，定义操作所有关系型数据库的统一标准
- 分类

分类	全称	说明
DDL	Data Definition Language	数据定义语言，用来定义数据库对象(数据库，表，字段)
DML	Data Manipulation Language	数据操作语言，用来对数据库表中的数据进行增删改
DQL	Data Query Language	数据查询语言，用来查询数据库中表的记录
DCL	Data Control Language	数据控制语言，用来创建数据库用户、控制数据库的访问权限

DDL -数据库

```
--查询所有数据库
show databases;
--查询当前数据库
select database();
--使用/切换数据库
use 数据库名;
--创建数据库
create database [if not exists] 数据库名 [default charset utf8mb4];
--删除数据库
drop database [if exists] 数据库名;
```

3.1.1 创建表

create table tablename{

字段1 字段类型【约束】[comment 字段1注释]

字段2 字段类型【约束】[comment 字段2注释]

}【comment 表注释】

【约束】作用：约束作用于表中字段上的规则，用于限制存储在表中的数据

约束	描述	关键字
非空约束	限制该字段值不能为null	not null
唯一约束	保证字段的所有数据都是唯一、不重复的	unique
主键约束	主键是一行数据的唯一标识，要求非空且唯一	primary key
默认约束	保存数据时，如果未指定该字段值，则采用默认值	default
外键约束	让两张表的数据建立连接，保证数据的一致性和完整性	foreign key

3.1.2 数据类型

类型	大小(byte)	有符号(SIGNED)范围	无符号(UNSIGNED)范围	描述	备注
tinyint	1	(-128, 127)	(0, 255)	小整数值	
smallint	2	(-32768, 32767)	(0, 65535)	大整数值	
mediumint	3	(-8388608, 8388607)	(0, 16777215)	大整数值	
int	4	(-2147483648, 2147483647)	(0, 4294967295)	大整数值	
bigint	8	(-2^63, 2^63-1)	(0, 2^64-1)	极大整数值	
float	4	(-3.402823466 E+38, 3.402823466351 E+38)	0 和 (1.175494351 E-38, 3.402823466 E+38)	单精度浮点数值	float(5,2): 5表示整个数字长度，2 表示小数位个数
double	8	(-1.7976931348623157 E+308, 1.7976931348623157 E+308)	0 和 (2.2250738585072014 E-308, 1.7976931348623157 E+308)	双精度浮点数值	double(5,2): 5表示整个数字长度，2 表示小数位个数
decimal				小数值(精度更高)	decimal(5,2): 5表示整个数字长度，2 表示小数位个数

选取原则：在满足业务需求的前提下，尽可能选择占用磁盘空间小的数据类型

age - tinyint unsigned

id -int unsighed

字符串类型

分类	类型	大小	描述		
字符串类型	char	0-255 bytes	定长字符串	char(10): 固定占用10个字符空间; 存储A, 占用10个空间; 存储ABC, 占用10个空间;	性能略高
	varchar	0-65535 bytes	变长字符串	varchar(10): 最多占用10个字符空间; 存储A, 占用1个空间; 存储ABC, 占用3个空间;	节约磁盘空间
	tinyblob	0-255 bytes	不超过255个字符的二进制数据		
	tinytext	0-255 bytes	短文本字符串	username varchar(50)	
	blob	0-65 535 bytes	二进制形式的长文本数据	idcard char(18)	
	text	0-65 535 bytes	长文本数据	phone char(11)	
	mediumblob	0-16 777 215 bytes	二进制形式的中等长度文本数据		
	mediumtext	0-16 777 215 bytes	中等长度文本数据		
	longblob	0-4 294 967 295 bytes	二进制形式的极大文本数据		
	longtext	0-4 294 967 295 bytes	极大文本数据		

日期类型

分类	类型	大小 (byte)	范围	格式	描述
日期类型	date	3	1000-01-01 至 9999-12-31	YYYY-MM-DD	日期值
	time	3	-838:59:59 至 838:59:59	HH:MM:SS	时间值或持续时间
	year	1	1901 至 2155	YYYY	年份值
	datetime	8	1000-01-01 00:00:00 至 9999-12-31 23:59:59	YYYY-MM-DD HH:MM:SS	混合日期和时间值
	timestamp	4	1970-01-01 00:00:01 至 2038-01-19 03:14:07	YYYY-MM-DD HH:MM:SS	混合日期和时间值，时间戳

设计步骤

- 阅读并分析页面原型及需求
- 分析表中包含哪些字段，以及字段的类型和约束
- 创建表结构

3.1.3 表结构的查询修改和删除

```
show tables; -- 查询当前数据库的所有表
desc 表名; -- 查询表结构
show create table 表名; -- 查询建表语句

alter table 表名 add 字段名 类型(长度) [comment 注释] [约束]; -- 添加字段
alter table 表名 modify 字段名 新数据类型(长度); -- 修改字段类型
alter table 表名 change 旧字段名 新字段名 类型(长度) [comment 注释] [约束]; -- 修改字段名与字段类型
alter table 表名 drop column 字段名; -- 删除字段
alter table 表名 rename to 新表名; -- 修改表名

drop table [if exists] 表名; -- 删除表
```

3.2 SQL语句-DML

用于对数据库中表的数据记录进行增删改操作

Insert

类型	SQL 语法
指定字段添加数据	INSERT INTO 表名 (字段名1, 字段名2) VALUES (值1, 值2);
全部字段添加数据	INSERT INTO 表名 VALUES (值1, 值2, ...);
批量添加数据（指定字段）	INSERT INTO 表名 (字段名1, 字段名2) VALUES (值1, 值2), (值1, 值2);
批量添加数据（全部字段）	INSERT INTO 表名 VALUES (值1, 值2, ...), (值1, 值2, ...);

Update

```
-- 修改数据
update 表名 set 字段名1 = 值1 , 字段名2 = 值2 , .... [ where  条件 ] ;
```

where 表示要更新的数据是哪一条；没有where就会更新整张表

delete

```
-- 删除数据
delete from 表名 [where 条件];
```

delete不能删除某一个字段的值；如果要这样操作可以使用update将该字段置为NULL

3.3 SQL语句-DQL

完整DQL语句语法

```
select
    字段列表
from
    表名列表
where
    条件列表
group by
    分组字段列表
having
    分组后条件列表
order by
    排序字段列表
limit
    分页参数
```

3.3.1 基本查询

```
-- 查询多个字段
select 字段1,字段2,字段3 from 表名;

-- 查询所有字段(通配符)
select * from 表名;

-- 为查询字段设置别名，as关键字可以省略
select 字段1 [as 别名1], 字段2 [as 别名2] from 表名;

-- 去除重复记录
select distinct 字段列表 from 表名;
```

3.3.2 条件查询

```
-- 条件查询
select 字段列表 from 表名 where 条件列表 ;
```

比较运算符	功能
>	大于
>=	大于等于
<	小于
<=	小于等于
=	等于
<> 或 !=	不等于
between ... and ...	在某个范围之内(含最小、最大值)
in(...)	在in之后的列表中的值，多选一
like 占位符	模糊匹配(_匹配单个字符，%匹配任意个字符)
is null	是null

逻辑运算符	功能
and 或 &&	并且（多个条件同时成立）
or 或 ||	或者（多个条件任意一个成立）
not 或 !	非，不是

判断字段值是不是null 应该用is null

between ... and ...　　应该先写最小值后写最大值

like +占位符 模糊匹配 （_ 表示单个字符　%表示匹配任意个字符）

组装多个查询条件　　使用and或者or

四.JDBC

JDBC:使用java操作关系型数据库的一套API（行为规范）

4.1 JDBC入门

- 注册驱动
- 获取连接
- 获取SQL语句并执行对象

- 执行SQL
- 释放资源

```
@Test
public void testUpdate() throws Exception {
    //1. 注册驱动
    //Class.forname将指定的类加载到内存中
    Class.forName("com.mysql.cj.jdbc.Driver");

    //2. 获取数据库连接
    String url = "jdbc:mysql://localhost:3306/web01";
    String username = "root";
    String password = "1234";
    Connection connection = DriverManager.getConnection(url, username, password);

    //3. 获取SQL语句执行对象
    Statement statement = connection.createStatement();

    //4. 执行SQL
    int i = statement.executeUpdate("update user set age = 25 where id = 1");//DML
    System.out.println("SQL执行完毕影响的记录数为: " + i);

    //5. 释放资源
    statement.close();
    connection.close();
}
```

4.2 JDBC查询数据

基于JDBC执行如下select语句，将查询结果封装到User对象中。

```
select * from user where username = 'daqiao' and password = '123456'
```

- ResultSet(结果集对象):

  ```
  ResultSet rs = statement.executeQuery()
  ```

  - `next()`：将光标从当前位置向前移动一行，并判断当前行是否为有效行，返回值为boolean。
    - `true`：有效行，当前行有数据
    - `false`：无效行，当前行没有数据

  `getXxx(...)`：获取数据，可以根据列的编号获取，也可以根据列名获取

  例如:

```
while (resultSet.next()) {
    int id = resultSet.getInt("id");
    //...省略
}
```

4.3 预编译SQL

- 优势1：防止SQL注入，更安全
    - SQL注入：通过控制输入来修改事先定义好的SQL语句，以达到执行代码对服务器进行攻击的目的
- 优势2：性能更高



执行SQL时会直接到缓存中去找编译后的SQL,无需再次编译

五.Mybatis

持久层框架，用于简化 JDBC的开发